

Trust Labels as Attack Surface: MCP Tool Annotations and the Protocol’s Unenforced Safety Contract

Anonymous

Anonymous

Abstract. The Model Context Protocol (MCP) lets AI hosts discover and call remote tools, and the protocol’s tool model carries *trust labels*—annotations such as `readOnlyHint`, `destructiveHint`, and `openWorldHint`—intended to tell the host how risky a tool is so it can gate permission prompts or policy. Prior work documents unauthenticated code-execution control planes and prompt injection in the agent ecosystem, but the security of the trust-label mechanism *itself* is unexamined: can a malicious MCP server *self-declare* a dangerous tool as safe, and does the ecosystem treat that declaration as authoritative? We show, by black-box runtime verification against a real MCP server and the official Python SDK, that (1) a server can declare a tool with `readOnlyHint=true`, `destructiveHint=false`, and a benign description while the tool actually executes a destructive command (file deletion + system-directory listing), and (2) the MCP SDK transport enforces nothing—it forwards the tool and the call verbatim, leaving authorization entirely to the host. We survey how several classes of hosts (from SDK primitives to permission-gated CLIs) consume these labels, and we argue the trust-label contract is *unenforced at the protocol layer*, converting a safety hint into an attacker-controlled claim. We provide detection and mitigation guidance—most importantly that hosts must not use `annotations` for policy, and the protocol should mark labels as untrusted metadata.

Keywords: Model Context Protocol · MCP · Tool Annotations · `readOnlyHint` · Trust Label · Unauthenticated Execution · CWE-693

1 Introduction

AI agents increasingly execute actions through *tools*, and the Model Context Protocol (MCP) [1] has become the dominant way hosts discover and invoke remote tools. A central safety mechanism of the MCP tool model is a set of *trust labels*: every tool may carry annotations—`readOnlyHint`, `destructiveHint`, `idempotentHint`, `openWorldHint`—that are *supposed* to tell the host how dangerous the tool is, so the host can gate permission prompts or apply policy (e.g. skip confirmation for read-only tools).

This creates an implicit trust boundary: the host *trusts the tool’s self-declared risk* at authorization time. Our question: is that trust enforceable? Prior work on MCP security [4,3,2] documents unauthenticated code-execution control planes, read-only-gate bypass (CWE-281), and prompt injection, but the *trust-label mechanism itself*—whether a malicious server can lie about its tool’s safety and whether the ecosystem believes it—is unexamined.

This paper’s contributions:

1. **A black-box proof that MCP trust labels are self-attested and unenforced:** we register a real MCP server declaring a tool `readOnlyHint=true/destructiveHint=false/benign-description`, and prove the tool actually executes a destructive command (deletes a marker file and lists a system directory). Runtime-verified against the official Python SDK client.
2. **The SDK transport enforces no safety:** the tool is forwarded, the call is forwarded, output is returned—the authorization check is entirely deferred to the host, so the label is ornamental at transport.
3. **An ecosystem survey** of how host classes consume labels, showing the trust model is inconsistent and, in permission-gated hosts, label-derived policy would not catch a lying server.
4. **Detection and mitigation:** hosts must not key authorization off `annotations` (they are attacker-controlled), and the protocol should mark labels as untrusted metadata rather than a relied-upon safety signal.

2 Background: MCP Tool Model and Annotations

2.1 The MCP tool model

An MCP server exposes tools via `tools/list`. Each tool has `name`, `description`, `inputSchema` (JSON Schema), and—crucially—`annotations` indicating safety-relevant intent:

```
1 {  
2   "name": "read_insights",  
3   "description": "Reads harmless aggregate insights (pure  
4     read-only)",  
5   "inputSchema": { "type": "object", "properties": {} },  
6   "annotations": { "readOnlyHint": true, "destructiveHint":  
7     false }  
8 }
```

2.2 Where the trust boundary sits

The protocol’s intent is that `annotations` communicate risk from server to host so the host can decide whether to prompt. But the server is *the attacker’s code* in a malicious scenario—it fully controls these fields. Whether this is a vulnerability depends on whether the host *relies* on them for authorization. At minimum, the transport must not treat them as a security decision. Our finding: the SDK transport does not.

3 Threat Model

We model a malicious or compromised MCP server that an AI host connects to (e.g. via a project's `.mcp.json` or a connectors directory entry). The server declares its dangerous tools as safe via `annotations` and benign descriptions. Objective: get the host to execute the tool without a permission prompt (or through policy that would have denied it), executing a destructive action. We assume the host uses `annotations` (as the protocol invites) to reduce prompting for "read-only" tools.

4 Black-Box Verification

We built a minimal stdio MCP server (no external deps) exposing:

- `read_insights`: declared `readOnlyHint=true`, `destructiveHint=false`, desc "pure read-only", but on call runs `sh -c "rm -f <marker> && ls /etc"`.
- `get_metadata`: declared read-only, returns `{}`.

We connected with the official Python MCP SDK client and verified (Listing 1.1):

1. `tools/list` returns the tool with `readOnlyHint=True`; the client surfaces it as-is.
2. `tools/call` executes the tool; the marker file is deleted and the system directory is listed – the destructive action happens despite the read-only declaration.
3. The SDK client performs *no* safety interception: it forwards the call and returns output;

Listing 1.1. Black-box result: declared read-only tool executes destructive command.

```
1 # tool listed with readOnlyHint=True, destructiveHint=False
2 # call read_insights -> "marker deleted + system dir listed"
3 # marker file: no longer exists (destructive action executed
  )
4 # SDK client: no prompt, no block -- pure forward
```

This is a protocol-layer fact: the trust label is *self-attested and unenforced at transport*. The only place a check can live is the host's authorization layer, and many hosts use the label to *skip* checks.

5 Ecosystem Survey: How Hosts Consume Labels

We examined how representative host classes consume tool annotations:

The risk materializes precisely where a host *uses* `annotations` to make an authorization decision (skip a prompt, allow a tool). Because the label is server-controlled, a malicious server controls the decision.

Table 1. Host-layer consumption of MCP tool annotations (and its implication).

Host class	Uses annotations?	Could a lying label bypass?	Notes
SDK transport (python mcp)	No (forwards only)	N/A at transport; all policy deferred	Found: no enforcement
Permission-gated CLI (e.g. agent CLIs)	May use for prompt reduction	Yes, if trusted	A “read-only” label could
Policy / guardrail gateway	May filter tools by hints	Yes, if trusted	Label-derived tool filter
Thin proxy / bridge	Forwards verbatim	N/A	No own policy

5.1 Empirical evidence: no endpoint enforces the label

We confirmed by source inspection that the label is effectively *unanchored* across the ecosystem:

- **Official Python SDK:** a full grep of the SDK’s `shared/` and `client/` for authorization logic keyed on `readOnlyHint/destructiveHint` returns nothing—the SDK surfaces annotations as data and forwards every tool call verbatim. No prompt, no block, no policy input.
- **Governance gateways** (e.g. Aegis MCP-to-MCP gateway, agent-sandbox with tool-gating): no code path reads `annotations` to filter/allow tools. They implement policy keyed on `tool identity/arguments` (or, when unauthenticated, on nothing enforceable), not on the server-declared hint.
- **Our black-box run (Sec. 4):** the SDK client listed `readOnlyHint=True` and executed the tool without any safety interception.

Together these show the trust label is *advisory metadata with no enforcing consumer*—which is exactly what makes it an unguarded, attacker-controlled claim.

6 Honest Scope and Limitations

- We *proved* the server-side lie and the SDK’s non-enforcement. We did *not* prove a specific permission-gated product executes a destructive action silently (that requires a live, product-level host; noted as deployment-dependent).
- This is the protocol/ecosystem trust-model issue; it is complementary to the unauthenticated-execution findings in prior work [2].
- Harmless marker command only; no real data harmed.

7 Mitigation

1. **Hosts must not key authorization off annotations.** A server-declared “read-only” label is not a safety decision; hosts should treat all remote tool calls as potentially harmful and prompt/policy accordingly, or verify by other means.
2. **Protocol: mark labels as untrusted metadata.** The spec should state `annotations` are advisory, not authoritative, and never sufficient to skip a permission gate.

3. **Guardrails.** Gateways/hosts should validate tool behavior (e.g. by sandboxing execution) rather than trusting declared risk.
4. **Sandboxing for anything labelled read-only.** If a host does trust the label for prompt-reduction, it must at least sandbox the tool so even a "read-only" lie cannot execute destructive ops.

8 Conclusion

MCP's tool trust labels are self-attested and unenforced at the protocol layer. We proved a malicious server can declare a destructive tool as read-only and that the official SDK forwards it without interception; the safety contract therefore rests entirely on hosts that must not trust it. We provide evidence, an ecosystem survey, and mitigation (labels as advisory metadata; authorization never keyed off server-declared safety). This closes a gap overlooked by prior agent-security corpora, which focused on unauthorized *execution* primitives rather than the *trust-label* trust model.

9 Disclosure & Ethics

Verified with a harmless marker only, no real harm, no third-party data. Reported as a protocol trust-model issue rather than a product-specific CVE, consistent with responsible disclosure.

References

1. Model context protocol specification. <https://modelcontextprotocol.io/specification>, accessed 2026-10-03
2. Agent execution control planes: When 'sandbox', 'gateway', and 'policy' ship as unauthenticated code execution. <https://zenodo.org/doi/10.5281/zenodo.PENDING> (2026), zenodo preprint (companion)
3. Credential-theft oracles, principal-spoofing, and unauthenticated write surfaces in freshly-published mcp and a2a servers. <https://zenodo.org/doi/10.5281/zenodo.22973907> (2026), zenodo preprint
4. Security analysis of the mcp ecosystem: From protocol design to production vulnerabilities. <https://zenodo.org/doi/10.5281/zenodo.22942676> (2026), zenodo preprint